

MemKernel: A Simple and Extensible Memory Service for Language Agents

Yicong Shi (985025074syc@gmail.com)

Abstract

Language agents require persistent memory to maintain user preferences, project decisions, and evolving facts across interactions. Storing complete conversation histories is simple, but it makes updates, retrieval, and provenance difficult to manage. We present **MemKernel**, a lightweight memory service that separates source events from durable memories. MemKernel uses a language model to extract atomic facts together with exact evidence spans, retrieves potentially related memories with vector similarity, and classifies each new fact as equivalent to, distinct from, or superseding an existing memory. The system stores active and historical memories in SQLite and retains links from every memory to its supporting source events. A tiered HOT/WARM/COLD lifecycle keeps logical validity separate from retrieval placement: recall searches HOT memories first, falls back to WARM, and can remove COLD vectors without deleting memory text or provenance. A REST API and an Agent Skill provide a small integration surface for language agents. In a preliminary retrieval-only evaluation on one LoCoMo conversation, MemKernel obtains an evidence hit rate of 83.33% and a mean evidence recall of 76.44% over 30 questions. These results are not an end-to-end question-answering evaluation or an evaluation of the tiering policy, but they indicate that a compact and auditable memory pipeline can recover useful evidence from long conversations.

Keywords: agent memory, long-term memory, language agents, tiered memory, provenance, conversational retrieval

1 Introduction

Large language models operate on bounded input contexts, while practical assistants may interact with a user for weeks or months. Repeatedly supplying the full interaction history increases cost and introduces irrelevant context. Retrieval-augmented generation addresses part of this problem by combining a language model with an external index (Lewis et al., 2020). Long-term agent memory introduces additional requirements: the system must decide what is worth storing, update information that changes, and explain which interaction supports a remembered fact.

Existing work has explored hierarchical context management and learned memory pipelines. MemGPT treats context management as a hierarchy of memory tiers and moves information between them as needed (Packer et al., 2023). Mem0 extracts and consolidates conversational information before retrieval, emphasizing scalable deployment (Chhikara et al., 2025). Benchmarks such as LoCoMo and LongMemEval show that long-term memory requires more than keyword matching because questions may involve multiple sessions, temporal relations, knowledge updates, and abstention (Maharana et al., 2024; Wu et al., 2025).

MemKernel is designed as a small, inspectable memory component rather than a complete conversational agent. Its central design choice is to keep four concerns separate: the original source event, the extracted durable fact, the logical state of that fact, and its physical retrieval tier. This separation makes it possible to retrieve compact memories, preserve evidence and update history, and control the active semantic working set without deleting the audit trail.

This paper makes four practical contributions:

1. It presents an evidence-bound extraction pipeline in which every generated memory must contain an exact supporting span from the current source event.
2. It introduces version-aware reconciliation with explicit EQUIVALENT, DISTINCT, and SUPERSEDES relations.
3. It separates logical memory state from a HOT/WARM/COLD retrieval lifecycle with hot-first recall, warm fallback, access-driven promotion, and non-destructive cold storage.
4. It provides a lightweight implementation based on FastAPI, SQLite, vector embeddings, and an Agent Skill integration layer.

2 Related Work

2.1 Retrieval and External Memory

Retrieval-augmented generation uses a non-parametric index to provide information that is not reliably accessible from model parameters alone (Lewis et al., 2020). This design supports inspection and updates, but a generic document index does not define which conversational facts should remain active when a user corrects or changes previous information.

MemGPT frames long contexts as a virtual memory problem and manages information across memory tiers (Packer et al., 2023). Mem0 instead builds a memory-centric pipeline that extracts, consolidates, and retrieves salient conversational information (Chhikara et al., 2025). MemKernel follows the extract-and-retrieve direction, but emphasizes a minimal service boundary, explicit memory states, and source-level provenance.

2.2 Long-Term Memory Evaluation

LoCoMo contains very long conversations spanning many sessions and evaluates question answering, summarization, and multimodal generation (Maharana et al., 2024). LongMemEval evaluates information extraction, multi-session reasoning, temporal reasoning, knowledge updates, and abstention (Wu et al., 2025). These benchmarks motivate memory systems that preserve temporal and update information rather than treating all past messages as equally current. The present work uses a small LoCoMo subset only as a preliminary retrieval check.

3 MemKernel

3.1 System Overview

MemKernel exposes two primary operations: *remember*, which processes a new source event, and *recall*, which retrieves relevant active or historical memories. Figure 1 summarizes the write and retrieval paths, including the shared provenance store. The complete processing path contains five stages.

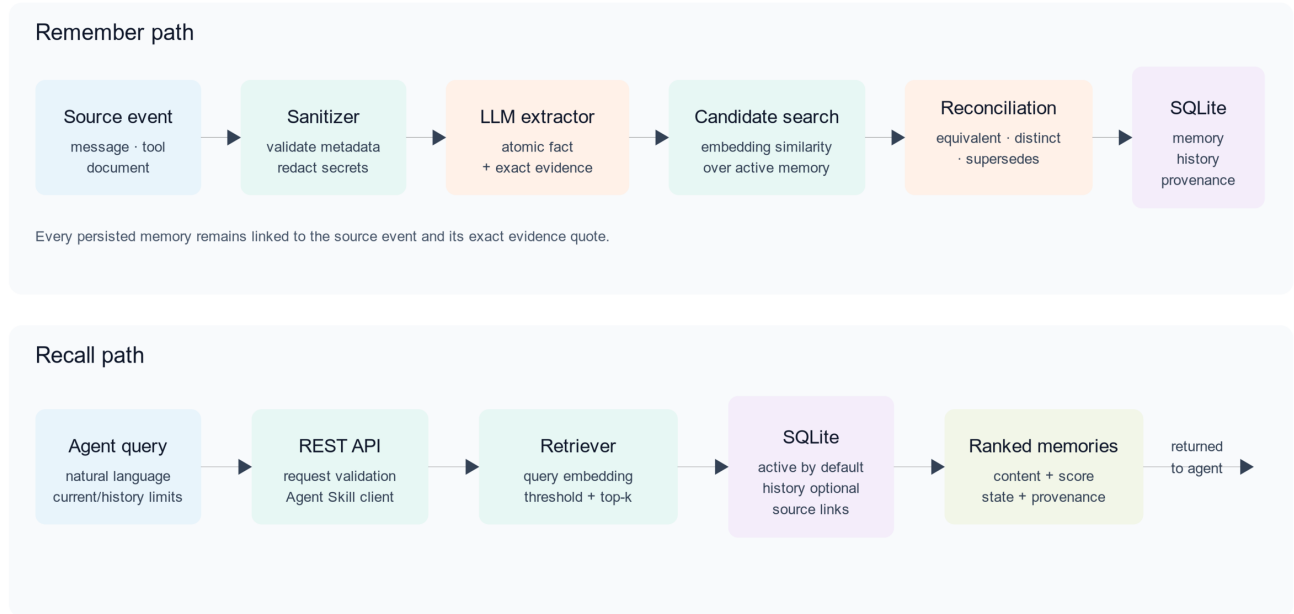


Figure 1. *MemKernel* system architecture. The remember path extracts and reconciles durable facts before an atomic write. The recall path searches the HOT working set first, falls back to WARM when needed, and retains history and provenance outside normal current-memory recall.

Stage	Input	Main operation	Output
Source processing	Message, tool output, or document	Validate metadata and redact common secret patterns	Sanitized source event
Fact extraction	Source event and optional recent context	Extract atomic facts and exact evidence spans	Candidate memories
Reconciliation	Candidate fact and similar active memories	Classify relation and apply a logical-state decision	Added, confirmed, or superseding memory
Retrieval	Natural-language query	Search HOT, fall back to WARM, rerank, and reinforce returned memories	Ranked current memories and optional history
Tier maintenance	Usage, confirmations, importance, and time	Promote, expire, demote, and optionally deindex	HOT/WARM/COLD placement

The implementation separates model providers from storage and retrieval interfaces. Language-model and embedding backends can therefore be replaced without changing the public memory API.

3.2 Evidence-Bound Extraction

A source event contains an identifier, content, source type, role, observation time, and JSON metadata. Supported source types are messages, tool outputs, and documents. Before extraction, MemKernel applies best-effort redaction to common private keys, bearer tokens, API keys, and named secret fields.

The extractor asks a language model to return a small list of atomic and durable facts. Each fact contains content and evidence. The evidence must be a non-empty exact substring of the sanitized source content; otherwise, the result is rejected. This validation does not prove that the extracted fact is a perfect interpretation, but it prevents the system from storing a fact with no textual support in the current event.

The extractor may receive up to six recent messages through metadata. This context is used only to resolve pronouns, ellipsis, confirmations, and corrections. A fact still needs evidence in the current source event. For example, if the previous message asks whether SQLite should be used and the current message says "Yes, use it," recent context can resolve the referent, while the confirmation remains the evidence.

3.3 Reconciliation and Logical Memory State

Extracted facts are compared with a small set of active memory candidates selected by embedding similarity. A language model assigns one of three relations:

- **EQUIVALENT**: the new fact expresses the same durable claim;
- **SUPERSEDES**: the new fact concerns the same mutable claim and makes the previous value outdated;
- **DISTINCT**: both facts can remain active.

These relations map to three storage actions, as shown in Figure 2. A distinct fact creates a new active memory. An equivalent fact produces no duplicate memory and links the new source as confirmation. A superseding fact creates a new active memory and marks the old record as superseded. The backend applies the source event, memory updates, embeddings, and provenance links in one SQLite transaction.

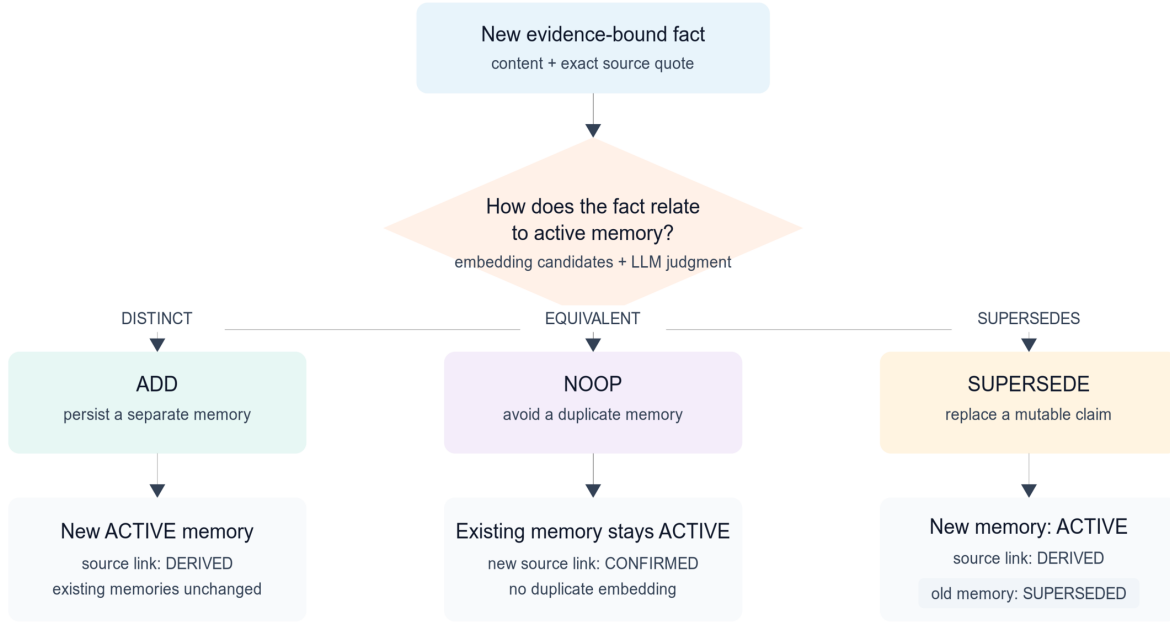


Figure 2. Reconciliation and logical memory state. Each extracted fact produces an explicit storage action while source links preserve whether the event derived or confirmed a memory.

Each memory record stores an identifier, content, creation time, state, and an optional pointer to its replacement. This representation retains a version chain without mixing outdated facts into default retrieval. Historical search remains available when an agent explicitly needs earlier values or the reason a value changed.

3.4 Provenance and Agent Integration

MemKernel stores source events separately from memories and connects them through a many-to-many provenance table. Each link includes the evidence quote and a link type: **DERIVED** for a source that created or replaced a memory, and **CONFIRMED** for a later source that matched an existing memory. An agent or developer can inspect both the complete memory history and the events supporting an individual memory.

The FastAPI service provides endpoints for remembering events, recalling memories, reading a memory’s history, and listing its sources. A bundled Agent Skill wraps these endpoints in a command-line client. Agents can therefore use MemKernel without embedding storage-specific logic in their prompts or application code.

3.5 Tiered Hot-Warm-Cold Memory

MemKernel separates logical truth from physical retrieval placement. **ACTIVE** and **SUPERSEDED** indicate whether a fact is current, while **HOT**, **WARM**, and **COLD** determine how aggressively it participates in semantic recall. A creation policy can set the initial tier, an importance value, an expiration time, and a pin flag. A separate usage record tracks access and confirmation counts together with their most recent timestamps.

Current-memory recall first retrieves **HOT** candidates. It queries **WARM** only when the number of eligible **HOT** results is smaller than the requested top_k . Candidate ordering uses a bounded combination of raw cosine similarity, importance, access history, confirmation history, pinning, and staleness, while the public retrieval score remains the raw cosine similarity. Memories returned to the caller record a new access, and returned or newly confirmed **WARM**

memories are promoted to **HOT**. Expired memories are excluded from semantic recall but remain available through administrative and provenance paths.

Lifecycle maintenance is explicit rather than hidden inside recall. The default policy demotes inactive **HOT** memories to **WARM** after 30 days and moves sufficiently stale, expired, or superseded memories to **COLD** after 180 days. Optional high and low watermarks can bound the **HOT** working set by demoting lower-importance and less-reinforced memories first. **COLD** memories retain text, history, and source links, but they do not participate in normal semantic recall; their embeddings are removed by default. Pinning protects memories from staleness- and capacity-based demotion, while explicit expiration remains authoritative.

Tier	Role	Default recall behavior	Automatic transition
HOT	Primary semantic working set	Searched first	Demoted by age or capacity maintenance
WARM	Searchable secondary set	Queried when HOT results are insufficient	Promoted on return or confirmation; demoted when stale
COLD	Non-destructive archival set	Excluded from normal semantic recall	Not automatically promoted

4 Preliminary Evaluation

4.1 Setup

We ran a retrieval-only experiment on one conversation from LoCoMo (Maharana et al., 2024). The run ingested 369 dialogue turns as 19 session-bounded chunks and evaluated 30 questions. Retrieval returned up to 20 active memories per question with the similarity threshold set to zero. Answer generation and LLM-based judging were disabled, so the experiment measures evidence retrieval rather than question-answering accuracy. The run does not isolate the tiered lifecycle and reports no maintenance, promotion, or cold-recovery comparison.

For each question, the benchmark maps retrieved memories back to their linked LoCoMo dialogue identifiers. **Evidence hit rate** is the proportion of questions for which at least one gold evidence identifier is recovered. **Evidence recall** is the fraction of gold evidence identifiers recovered for a question, averaged across questions with annotated evidence.

4.2 Results

Metric	Result
Questions	30
Evidence hit rate	83.33%
Mean evidence recall	76.44%
Average retrieval latency	32.79 ms

The result shows that the retrieved memories frequently preserve links to relevant source turns. Provenance is important for this measurement: the benchmark can evaluate evidence coverage without relying on lexical overlap between an extracted memory and a reference answer.

The numbers should be interpreted as a feasibility result. The run covers one conversation, uses a permissive zero similarity threshold, and does not compare against alternative memory systems. It also does not test whether a downstream language model can combine the retrieved memories into a correct answer.

5 Discussion

MemKernel’s main benefit is not a new retrieval algorithm, but a compact composition of extraction, updating, tiered retrieval, and auditing. Evidence validation limits unsupported memory creation, explicit logical states keep outdated information out of normal recall, and provenance links allow developers to inspect why a memory exists. The HOT/WARM/COLD lifecycle narrows routine vector search and vector storage without erasing the evidence needed for later inspection. The service boundary also makes the component usable by different agents and model providers.

The explicit maintenance clock and configurable high and low watermarks make lifecycle experiments reproducible. At the same time, separating the public similarity score from the internal lifecycle-aware rank score preserves compatibility for callers that already interpret retrieval scores as cosine similarity.

The design trades breadth for simplicity. SQLite is appropriate for a local or single-node deployment but does not provide distributed

scaling. Reconciliation depends on both embedding quality and a language-model judgment, so errors can merge distinct facts or preserve contradictory facts. Regex-based secret redaction is best-effort and cannot replace application-level access control. The current API also requires deployment-specific authentication and authorization before it is exposed beyond a trusted local environment.

6 Limitations

The evaluation is preliminary and insufficient for claims of state-of-the-art performance. It includes only one LoCoMo conversation and 30 questions, omits full question answering, and reports no statistical uncertainty or baseline comparison. The retrieval latency is specific to the evaluated local configuration and should not be treated as a general performance guarantee. Future evaluation should cover all LoCoMo conversations, LongMemEval, stronger memory baselines, multiple model providers, end-to-end answer accuracy, and ablations of extraction context, reconciliation, and provenance-aware retrieval.

MemKernel currently represents durable information as natural-language facts. This improves readability but may be less suitable than structured graphs for relational or highly temporal queries. The three-way reconciliation label set also simplifies cases where information is partially compatible, uncertain, or valid only during a bounded time interval.

The tiered lifecycle bounds the searchable vector set when maintenance is configured, but it deliberately retains memory rows, source events, and provenance. Total database size can therefore continue to grow. The 30-day and 180-day thresholds, reranking weights, and capacity policy are currently heuristics rather than empirically calibrated values. Because COLD memories are excluded from default recall and are not automatically reactivated, aggressive demotion may hide relevant long-tail information. Future evaluation should measure recall, latency, and storage under flat, static-tier, and adaptive-tier policies, and should study controlled rehydration and deletion mechanisms.

7 Conclusion

This paper introduced MemKernel, a small and extensible memory service for language agents. The system combines evidence-bound fact extraction, version-aware reconciliation, tiered HOT/WARM/COLD retrieval, and source provenance in a SQLite-backed service. A preliminary LoCoMo experiment suggests that the retrieval pipeline can recover relevant evidence from a long conversation, while the effectiveness of the new tiering policy and the end-to-end system remain future evaluation targets. MemKernel provides a practical foundation for studying agent memory without requiring memory management to be tightly coupled to a specific agent framework.

References

- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. *arXiv preprint arXiv:2504.19413*. <https://doi.org/10.48550/arXiv.2504.19413>.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems* 33.
- Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. Evaluating Very Long-Term Conversational Memory of LLM Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13851-13870. <https://doi.org/10.18653/v1/2024.acl-long.747>.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2023. MemGPT: Towards LLMs as Operating Systems. *arXiv preprint arXiv:2310.08560*. <https://doi.org/10.48550/arXiv.2310.08560>.
- Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2025. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. In *The Thirteenth International Conference on Learning Representations*.